# Аутентификация во frontend-приложении

Навигация, защищённые разделы и роли в Internet Shop

# Цель занятия

- Научиться переключать разделы одной HTML-страницы без перезагрузки.
- Показывать страницы и пункты меню с учётом входа пользователя и его роли.
- Использовать готовые login(), apiFetch() и authState.js из ЛР-5, не переписывая их.

# Учебные вопросы

| № | Учебный вопрос | Что сделаем |
|---|---|---|
| 1 | Состояние входа и разделы интерфейса | Определим, что видят гость, пользователь и администратор |
| 2 | Hash Routing: навигация без перезагрузки | Свяжем адрес #/... с HTML-разделом |
| 3 | Protected Pages и Route Guards | Ограничим открытие профиля и панели администратора |
| 4 | Связь навигации с login и logout | Обновим меню и проверим поведение приложения |

# Что уже сделано в лабораторной №5

- Frontend существует в папке frontend/: index.html, app.js, api.js, authState.js, config.js, styles.css.
- login() отправляет POST /api/auth/login; apiFetch() подставляет Bearer JWT.
- authState.js хранит user и accessToken в памяти вкладки.
- Сервер Express отдаёт страницу по http://localhost:3000/.

# 1. Состояние входа и разделы

- Сначала решим, какие разделы нужны и как определить пользователя.

# Термины: guest, user и admin

- Guest (гость) — посетитель, который ещё не вошёл в систему.
- Authenticated user — пользователь, успешно выполнивший вход.
- Role (роль) — значение user.role; в нашем проекте это user либо admin.
- Auth state — сведения о входе, которые frontend использует для отображения интерфейса.

# Auth state machine: loading → guest или authenticated

При запуске приложение определяет состояние аутентификации и показывает соответствующий интерфейс

**Старт приложения**

Открываем приложение в браузере

**loading**

определение Authentication

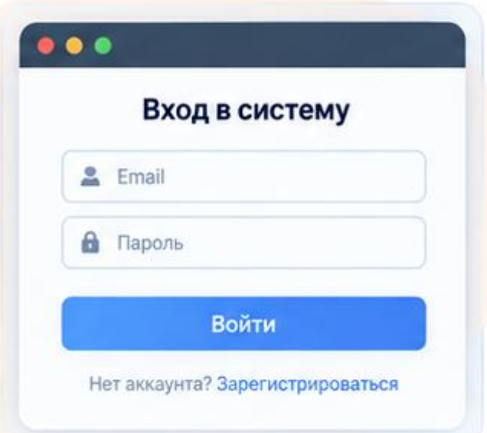
Есть действительная Authentication?

?

Нет

**guest**

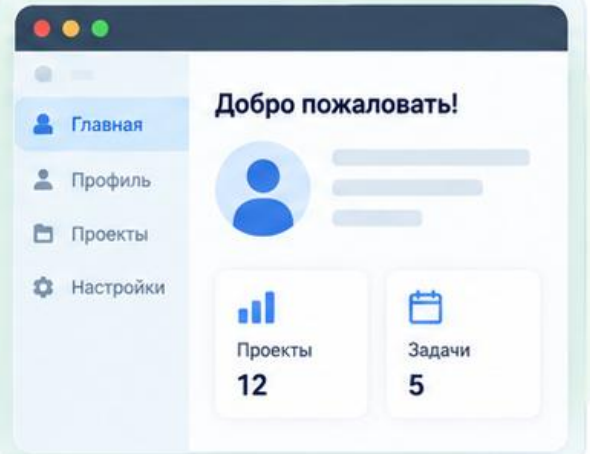
- не аутентифицирован
- показать login / public UI




Да

**authenticated**

- User подтверждён
- показать protected UI




*Один запуск — два возможных сценария интерфейса*

# Используем готовый authState.js

```javascript
import { getCurrentUser } from "./authState.js";

const user = getCurrentUser();
console.log(user?.name);
console.log(user?.role);
```

**Что важно**

- getCurrentUser() возвращает объект пользователя либо null.
- Оператор ?. безопасно обращается к свойству, когда user может быть null.
- Повторно создавать authState.js не нужно.

# Какие разделы показывать

| Раздел | Гость | user | admin |
|---|---|---|---|
| Каталог | Да | Да | Да |
| Вход | Да | Нет | Нет |
| Профиль | Нет | Да | Да |
| Панель администратора | Нет | Нет | Да |

# Почему одного hidden недостаточно для навигации

- В ЛР-5 мы показывали и скрывали guest-section и user-section через hidden.
- Если разделов становится больше, удобно связывать каждый с отдельным адресом в браузере.
- При этом HTML-страница остаётся той же: мы только меняем видимый раздел.

# 2. Hash Routing: навигация без перезагрузки

- Разберём адреса с символом # до использования location.hash в коде.

# Новое понятие: Hash Routing

- Hash Routing — способ навигации внутри frontend через фрагмент URL после символа #.
- Пример: http://localhost:3000/#/profile — раздел «Профиль».
- Фрагмент #/profile не передаётся Express как путь HTTP-запроса.
- JavaScript браузера читает фрагмент и показывает соответствующий раздел.

# Адреса Internet Shop

| Адрес в браузере | Раздел |
| --- | --- |
| http://localhost:3000/#/catalog | Каталог |
| http://localhost:3000/#/login | Вход |
| http://localhost:3000/#/profile | Профиль |
| http://localhost:3000/#/admin | Панель администратора |

# Как изменить раздел с помощью location.hash

```javascript
location.hash = "#/login";

console.log(location.hash); // "#/login"
```

**Что важно**

- location.hash — свойство браузера, содержащее фрагмент URL с символом #.
- Присваивание меняет адрес без полной перезагрузки страницы.
- Само присваивание не показывает форму: для этого нужен код переключения разделов.

# Событие hashchange

```javascript
window.addEventListener("hashchange", () => {
  console.log("Новый раздел:", location.hash);
});
```

**Что важно**

- hashchange возникает при изменении фрагмента URL.
- Обработчик узнаёт, какой раздел пользователь выбрал.
- При первой загрузке обработчик hashchange сам по себе не запускается — нужную функцию вызывают отдельно.

# HTML: связываем разделы с адресами

HTML

```
<nav>
  <a href="#/catalog">Каталог</a>
  <a href="#/profile">Профиль</a>
  <a href="#/admin">Администрирование</a>
</nav>

<section data-page="#/catalog">Каталог</section>
<section data-page="#/login" hidden>Вход</section>
<section data-page="#/profile" hidden>Профиль</section>
<section data-page="#/admin" hidden>Панель</section>
```

**Что важно**

- href с # меняет фрагмент URL.
- data-page — пользовательский HTML-атрибут; в нём указан адрес соответствующего раздела.
- Это схема добавления разделов к уже существующему index.html, а не замена всей страницы.

# showPage(): показываем выбранный раздел

```javascript
function showPage(page) {
  document.querySelectorAll("[data-page]").forEach(section => {
    section.hidden = section.dataset.page !== page;
  });
}
```

**Что важно**

- querySelectorAll() выбирает все элементы с атрибутом data-page.
- section.dataset.page читает значение этого атрибута.
- hidden = true скрывает все разделы, кроме запрошенного.

# renderPage(): соединяем URL и страницу

```javascript
function renderPage() {
  const page = location.hash || "#/catalog";
  showPage(page);
}

window.addEventListener("hashchange", renderPage);
renderPage();
```

**Что важно**

- renderPage() читает текущий адрес и вызывает showPage().
- Обработчик hashchange нужен для переходов; явный renderPage() — для первого открытия.
- Проверку доступа добавим на следующем учебном вопросе.

# 3. Protected Pages и Route Guards

- Теперь ограничим доступ к отдельным frontend-разделам.

# Два новых термина

- Protected Page — раздел интерфейса, который отображается только после проверки состояния входа. Пример: профиль.
- Route Guard — функция, проверяющая, разрешён ли переход на выбранный раздел.
- Guard управляет интерфейсом и навигацией; он не заменяет проверку JWT и роли на backend.

# requireUser(): проверяем вход

```javascript
function requireUser() {
  const user = getCurrentUser();

  if (!user) {
    location.hash = "#/login";
    return false;
  }
  return true;
}
```

**Что важно**

- Если пользователя нет, браузер переходит на #/login.
- false запрещает показывать запрошенную защищённую страницу.
- Здесь используется getCurrentUser() из ЛР-5.

# requireAdmin(): проверяем роль

```javascript
function requireAdmin() {
  const user = getCurrentUser();
  if (!user) {
    location.hash = "#/login";
    return false;
  }
  if (user.role !== "admin") {
    location.hash = "#/catalog";
    alert("Недостаточно прав");
    return false;
  }
  return true;
}
```

**Что важно**

- Гость сначала направляется на форму входа.
- У вошедшего пользователя проверяется поле role.
- При роли user раздел администратора не открывается; настоящие права проверяет Express.

# Добавляем guards в renderPage()

```javascript
function renderPage() {
  const page = location.hash || "#/catalog";

  if (page === "#/profile" && !requireUser()) return;
  if (page === "#/admin" && !requireAdmin()) return;

  showPage(page);
}

window.addEventListener("hashchange", renderPage);
renderPage();
```

**Что важно**

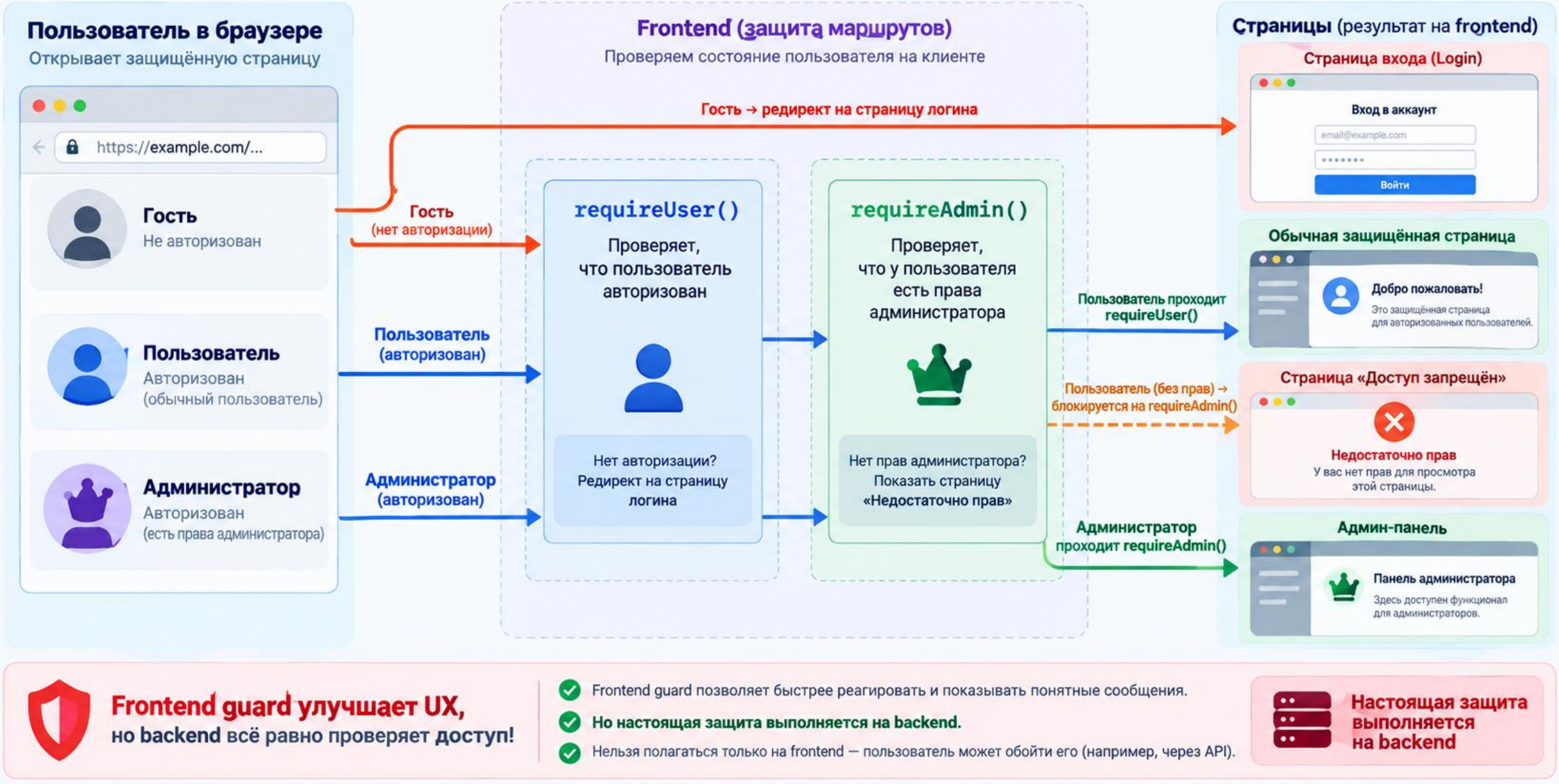
- Это итоговая версия renderPage(): она заменяет упрощённую функцию из вопроса 2.
- При запрете перехода guard меняет hash, и следующий hashchange покажет разрешённый раздел.
- Неизвестные адреса в учебном примере лучше дополнительно направлять на каталог.

# Почему route guard не защищает данные

- Frontend-код выполняется в браузере и может быть изменён пользователем.
- Можно отправить запрос к /api/products напрямую через Postman или curl, минуя меню.
- Поэтому requireAuth и requireRole("admin") в Express остаются обязательными.
- Скрытая кнопка и блокировка страницы улучшают интерфейс, но не обеспечивают безопасность API.

# Protected page guard: guest → login, user → page, admin → admin page

Как работает защита страниц на frontend (например, в React / Vue / Angular)




**Frontend guard улучшает UX, но backend всё равно проверяет доступ!**

- Frontend guard позволяет быстрее реагировать и показывать понятные сообщения.
- **Но настоящая защита выполняется на backend.**
- Нельзя полагаться только на frontend — пользователь может обойти его (например, через API).

**Настоящая защита выполняется на backend**

# 4. Связь навигации с login и logout

- Дополним готовые функции ЛР-5 отображением страниц и меню.

# Меню зависит от роли

```javascript
function renderNavigation() {
  const user = getCurrentUser();

  loginLink.hidden = Boolean(user);
  profileLink.hidden = !user;
  adminLink.hidden = user?.role !== "admin";
  logoutButton.hidden = !user;
}
```

**Что важно**

- loginLink, profileLink, adminLink и logoutButton — ссылки/кнопки, найденные в DOM заранее.
- Гость видит «Вход», обычный user — «Профиль», admin — ещё и панель.
- При смене состояния входа функцию вызывают повторно.

# После login: обновляем страницу и меню

JAVASCRIPT

```
// В обработчике формы из ЛР-5
const user = await login(emailInput.value, passwordInput.value);
renderNavigation();
location.hash = "#/profile";
renderPage();
```

**Что важно**

- login() уже получает JWT и сохраняет сведения пользователя через setAuth().
- renderNavigation() показывает доступные пункты меню.
- location.hash выбирает профиль; renderPage() обновляет содержимое даже при повторном выборе адреса.

# После logout: возвращаемся к каталогу

```javascript
// В обработчике кнопки «Выйти»
clearAuth();
renderNavigation();
location.hash = "#/catalog";
renderPage();
```

**Что важно**

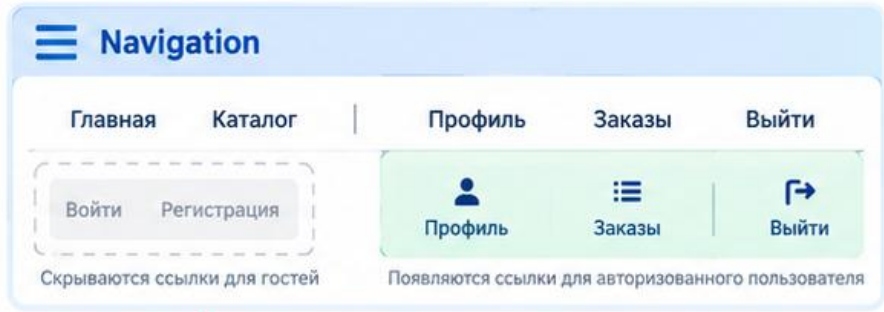
- clearAuth() из ЛР-5 очищает user и JWT в памяти.
- Меню перестраивается для гостя; защищённый раздел больше не показывается.
- Для демонстрации можно использовать уже существующий logout-обработчик.

# Проверка работы интерфейса

| Сценарий | Ожидаемый результат |
| --- | --- |
| Гость открывает #/profile | Переход на #/login |
| user открывает #/profile | Профиль отображается |
| user открывает #/admin | Отказ и переход на каталог |
| admin открывает #/admin | Панель отображается |
| Пользователь нажимает «Выйти» | Открывается каталог, меню гостя |

# Frontend auth lifecycle: login → state → protected request → response

От входа пользователя до обновления интерфейса




Интерфейс меняется после login

### Login

Email

user@example.com

Password

••••••••

Войти

Пользователь вводит credentials

### POST /api/auth/login

```
fetch('/api/auth/login', {
  method: 'POST',
  headers: {
    'Content-Type':
     'application/json'
  },
  body: JSON.stringify({
    email, password
  })
})
```

Frontend отправляет login request

### Ответ backend

```
{
  "user": {
    "id": 1,
    "name": "Иван",
    "email": "user@example.com"
  },
  "accessToken":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6…"
}
```

Backend возвращает пользователя и access token

### authState

```
{
  status: 'authenticated',
  user: {
    id: 1,
    name: 'Иван',
    email: 'user@example.com"
  },
  accessToken:
  'eyJhbGciOiJIUzI1NiIsInR5cCI6…"
}
```

Frontend сохраняет auth state

### Protected request

GET /api/orders/my

```
apiFetch('/api/orders/my',{
  method: 'GET',
  headers: {
    'Authorization':
    'Bearer <accessToken>'
  }
})
```

apiFetch() добавляет Bearer token

### Protected API

Express JS

GET /api/orders/my (protected)

- Проверяет JWT
- Извлекает данные пользователя
- Проверяет права доступа

Backend проверяет JWT

### JSON Response

```
[
  {
    "id": 101,
    "title": "Ноутбук",
    "status": "Доставлен",
    "total": 59900
  },
  {
    "id": 102,
    "title": "Клавиатура",
    "status": "В пути",
    "total": 8900
  }
]
```

Данные возвращаются с сервера

### UI

Мои заказы

- Ноутбук 59 900 ₽ — Доставлен
- Клавиатура 8 900 ₽ — В пути
- Наушники 4 500 ₽ — Обработка

Frontend обновляет DOM / UI

**Ключевая идея:**

1. Login создаёт authenticated state
2. **authState** управляет navigation и protected requests
3. **Backend** остаётся источником истины

# Что происходит после F5

- JWT и пользователь из authState.js в нашем проекте хранятся только в памяти JavaScript.
- После перезагрузки страницы эти значения сбрасываются: пользователь снова становится гостем.
- Auth Bootstrap — проверка действующей серверной сессии при запуске приложения. Для нашей реализации без сохраняемой сессии он не требуется.
- HttpOnly cookies и восстановление сессии — отдельная дополнительная тема, не задача этой лабораторной.

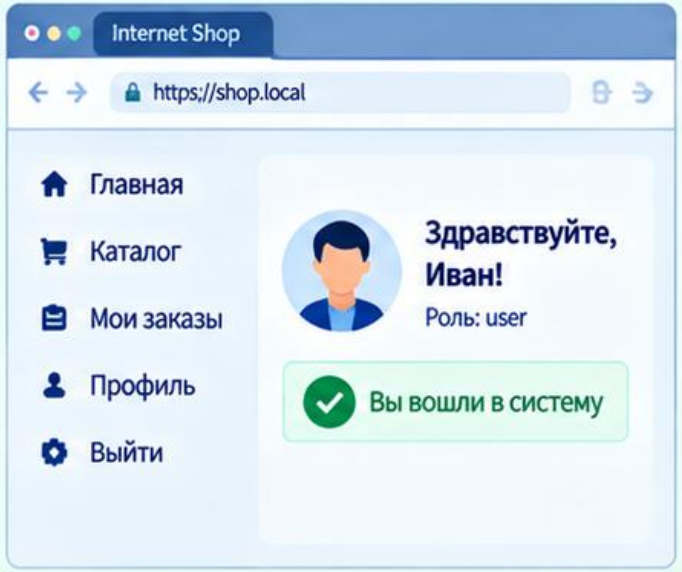
# Reload: in-memory JWT исчезает → frontend снова guest

Токен хранится только в памяти JavaScript. После перезагрузки страницы (F5) состояние приложения создаётся заново.

Internet Shop
Learn. Build. Deploy.

## 1 До перезагрузки страницы

Пользователь аутентифицирован




**Состояние во фронтенде (в памяти JavaScript)**

```
authState = {
  user: {
    id: 1,
    name: "Иван",
    role: "user"
  },
  accessToken: "eyJhbGciOi..."
}
```

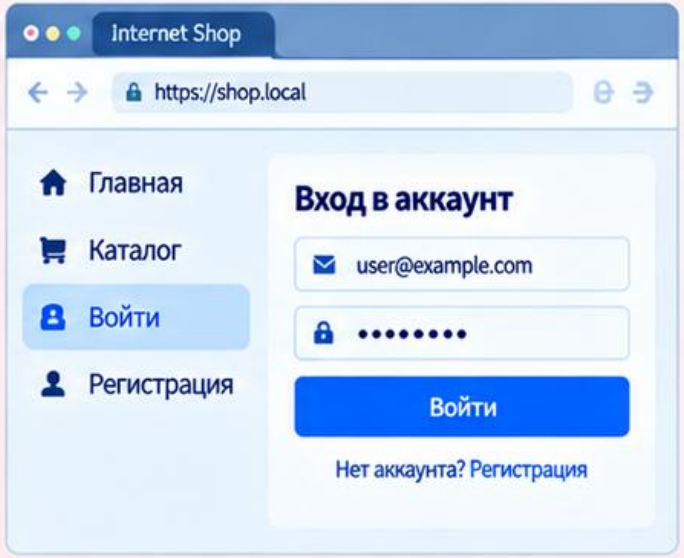
RAM **Хранится в памяти страницы**
(Heap / JavaScript state)

**Перезагрузка страницы**

F5
или
Ctrl + R

## 2 После перезагрузки страницы

Приложение снова в состоянии guest




**Состояние во фронтенде (в памяти JavaScript)**

```
authState = {
  user: null,
  accessToken: null
}
```

RAM **Память страницы очищена**
(новый JavaScript context)

### Что работает сейчас

- Доступ к защищённым страницам (например, /orders)
- Запросы к API с заголовком `Authorization: Bearer <token>`
- Отображение данных пользователя

### Что изменилось

- Токен, хранившийся в памяти, потерян
- authState создан заново (`user = null`)
- Приложение снова в состоянии guest
- Для доступа к защищённым страницам нужно снова войти в систему

### Почему так происходит?

- Все переменные JavaScript хранятся в памяти текущей страницы (browser tab).
- При перезагрузке (F5 / Ctrl + R) текущий JavaScript context уничтожается.
- Создаётся новая страница с чистым состоянием (authState = guest).
- Токен в памятя не сохраняется между сессиями.

### Важно

- Такое поведение — это ожидаемо и безопасно.
- Если нужно сохранять сессию между перезагрузками, используйте другой механьим (например, HttpOnly cookie + /auth/me).
- Не сохраняйте JWT в localStorage — это уязвимо к XSS.

### Рекомендация

Для учебного проекта мы храним токен только в памяти страницы. После перезагрузки пользователь входит заново. Для «запоминающейся» сессии рассмотрим cookie-based архитектуру на следующей лекции.

# Практика для Internet Shop

- Используйте существующие frontend/index.html и frontend/app.js из ЛР-5.
- Добавьте разделы catalog, login, profile, admin и ссылки с #/.
- Реализуйте showPage(), renderPage(), requireUser(), requireAdmin() и renderNavigation().
- Не переписывайте apiFetch(), login() и authState.js; серверные проверки прав сохраняются.

# Итоги занятия

1. Hash Routing связывает фрагмент URL после # с разделом одной HTML-страницы.
2. showPage() переключает блоки, а renderPage() реагирует на текущий адрес.
3. requireUser() и requireAdmin() управляют доступом к разделам интерфейса.
4. renderNavigation() обновляет меню после login/logout.
5. Backend продолжает самостоятельно проверять JWT и права доступа.

# Контрольные вопросы

1. Чем отличается маршрут Express /api/products от frontend-адреса #/profile?
2. Зачем после подписки на hashchange отдельно вызывать renderPage()?
3. Что проверяют requireUser() и requireAdmin()?
4. Почему скрытие admin-раздела не заменяет requireRole("admin") на сервере?
5. Что должно произойти с интерфейсом после login и logout?
6. Почему после F5 в нашей учебной реализации требуется повторный вход?